TECHNICAL HANDOVER

2B Environmental

Waste-Management Operations & Career Portal Platform

Laravel 12 · PHP 8.2 · MySQL

DOCUMENT	Engineering Handover & System Reference
PREPARED	7 October 2026
CODEBASE BRANCH	new-career-app
AUDIENCE	Incoming engineers / maintainers
SCOPE	Full application: architecture, modules, flows, data model, risks

Contents

1. System Overview

2. Technology Stack & Environment

3. Application Architecture

4. Authentication & Authorization (RBAC)

5. Module — Waste Disposal

6. Module — Cesspool & Septic Inspections

7. Module — Document Management & E-Signature

8. Module — Announcements

9. Module — Community Calendar

10. Module — Incident Reports

11. Module — Employees (HR) & Employee Portal

12. Notifications

13. Activity Log & Auditing

14. Email & Scheduled Jobs

15. Database Schema Summary

16. File Storage Conventions

17. Known Issues, Risks & Handover Flags

18. Setup & Deployment Notes

1. System Overview

"2B Environmental" is an internal web platform for a Hawai‘i-based waste-management / wastewater-pumping company. It bundles two distinct applications into a single Laravel codebase:

- **Admin back office** (`/dashboard` and below) — staff manage waste-disposal records, cesspool & septic inspection reports, users/roles/permissions, an HR & career portal (employees, documents, announcements, calendar, incident reports), activity logs and notifications.
- **Public / employee frontend** — public intake forms for field submissions (waste disposal, cesspool, septic), plus a self-service **Employee Portal** (login, document library with read-and-sign, incident reporting, announcements, calendar, notifications, profile).

Both halves share one `users` table and one Laravel `web` session guard. A user's **role slug** determines which half they belong to — a user whose role slug is `employee` is an Employee-Portal user; everyone else is back-office staff. There is no separate authentication guard for employees.

Business domain	What the platform does
Wastewater pumping & hauling	Captures waste-disposal pickups and generates the official County of Hawai‘i monthly source report.
Cesspool & septic inspections	Field inspection intake forms with media (photos/video) & signatures; admin review, PDF export, email delivery.
HR / career portal	Employee onboarding (welcome emails), document distribution with mandatory read-and-sign acknowledgment, announcements, team calendar, incident reporting.
Governance	Granular role-based permissions, full activity audit log, in-app notifications.

2. Technology Stack & Environment

Layer	Choice
Framework	Laravel 12 (PHP `^8.2`)
Database	MySQL (local DB name `2be`)
Queue	`database` driver (note: mail is sent *synchronously*, not queued)
Mail	SMTP (Gmail, `smtp.gmail.com:587` TLS)
PDF generation	`barryvdh/laravel-dompdf` (dompdf) — plus a Chrome/Edge headless "print-to-pdf" path for signature certificates, falling back to dompdf. `setasign/fpdf` + `fpdi` are in `composer.json` but **not actually used** anywhere.
Frontend build	Vite 7 + Tailwind CSS 4 (minimal — the app is predominantly server-rendered Blade over a Bootstrap-based admin theme with DataTables; no SPA framework)
Local host	XAMPP on Windows (`c:\xampp\htdocs\2be`)

> **ENVIRONMENT NOTE**
> The local `.env` runs with `APP_DEBUG=true` and `APP_ENV=local`, and `.env` is tracked in git. Ensure production flips `APP_DEBUG=false` and uses a separate, untracked secrets file.

3. Application Architecture

3.1 Request routing

All routes live in `routes/web.php` (255 lines). Two groups:

- **Admin group** — `middleware(['auth:web', PreventBackHistoryMiddleware])`. Every route is additionally gated by a granular `permission:<slug>` middleware (e.g. `permission:manage-disposal-details.view`).
- **Frontend group** — `middleware([PreventBackHistoryMiddleware])`; public forms and the employee portal. Logged-in-only employee routes add the `employee.auth` middleware.

3.2 Key directories

Path	Contents
`app/Http/Controllers/Backend/`	15 admin controllers (Disposal, Cesspool, Septic, Role, User, Permission, Employee, Document, DocumentCategory, Announcement, IncidentReport, CalendarEvent, ActivityLog, Notification, EmailSettings)
`app/Http/Controllers/Frontend/`	Home, Cesspool, Septic, Employees (the 801-line portal controller)
`app/Http/Middleware/`	`CheckPermission`, `EmployeeAuth`, `PreventBackHistoryMiddleware`
`app/Models/`	17 Eloquent models + `Concerns/TracksDeletedBy` trait
`app/Services/`	`DocumentSignatureService` (certificate PDF generation)
`app/Observers/`	`ActivityObserver` (audit logging)
`app/Mail/`, `app/Console/Commands/`	Monthly-report reminder mailable + command
`resources/views/backend|frontend|pdf|emails/`	Blade templates (admin theme, public forms, PDF layouts, mail bodies)
`public/uploads/`	User-uploaded files (documents, avatars, incident media, signatures, announcements, calendar flyers)

3.3 Cross-cutting conventions

- **Soft deletes everywhere.** Nearly all models use `SoftDeletes` + a `TracksDeletedBy` trait that stamps `deleted_by` with the acting user's id on soft-delete (skips force-deletes and unauthenticated deletes).
- **Audit by observer.** An `ActivityObserver` is attached to 12 models and records create/update/delete/restore events into `activity_logs` (console/CLI operations are skipped).
- **Uploads under `public/`.** Most files are written directly into `public/uploads/...` and referenced by a relative path. (Exception: cesspool/septic inspection *images* use the Laravel public storage disk — see §16.)

4. Authentication & Authorization (RBAC)

4.1 Data model

Custom role-based access control built on the standard session `web` guard. A `User` belongs to one `Role` (`role_id`, nullable). A `Role` has many `Permission`s through the `permission_role` pivot (composite PK, cascade delete). Permission slugs follow a `module.action` convention (e.g. `users.create`, `documents.edit`, `permissions.assign`).

Model	Key fields	Notes
`Role`	name, slug (unique), description, is_protected, is_active	Constant `SUPERADMIN_SLUG = 'superadmin'`. Caches its permission slugs per instance.
`Permission`	name, slug (unique), module, description	Slug is never changed on edit (route middleware references depend on it).
`User`	name, email, phone, avatar, password, role_id, is_active, welcome_email_sent_at	Casts password→hashed, is_active→bool.

4.2 Permission resolution

The `permission:<slug>` middleware (`CheckPermission`) runs on each admin route:

1. No authenticated user → redirect to `admin.login`.
2. User exists but `is_active = false` → force logout, invalidate session, redirect with "account deactivated".
3. `User::hasPermission($slug)` fails → `abort(403)`.

`User::hasPermission()` logic (in order): inactive user → `false`; **superadmin → `true` (bypasses all checks)**; no role or inactive role → `false`; otherwise delegates to `Role::hasPermission()` which checks the role's cached permission-slug set.

> **SUPERADMIN**
> Superadmin is protected two ways: (1) it implicitly holds every permission regardless of pivot rows; (2) the `is_protected` flag blocks deactivating the role, deleting it, or editing its permission matrix. `UserController` also refuses to delete the last remaining superadmin user. Caveat: the superadmin bypass is *active-account gated* — deactivating a superadmin *user* strips their access because the `is_active` check runs first.

4.3 Admin login flow (`LoginController`)

1. POST `/login` → validate email+password → `Auth::attempt()` on the `web` guard with optional "remember".
2. If the authenticated user has role slug `employee` → immediately logged back out and redirected to admin login (employees must use the Employee Portal).
3. On success: regenerate session, write a `login` ActivityLog entry, redirect to `admin.dashboard`.
4. Logout is a GET `/logout` route that flushes the session and writes a `logout` log.

4.4 Role / User / Permission management

- **Roles** (`RoleController`): CRUD with auto-generated unique slug. Protected roles can't be deactivated/deleted; a role with assigned users can't be deleted; slug is never changed on update.

- **Users** (`UserController`): manages non-employee staff only (employees are excluded from the list and can't be assigned here — they're created in the HR section). Guards: can't deactivate/delete your own account; can't delete the only superadmin.
- **Permission matrix** (`PermissionController`): per-role grid grouped by module; `update()` does a full `sync()` of pivot rows (protected roles blocked). A **permission catalog** lets admins add/edit/remove permission definitions as new tabs appear (slug preserved on edit).

4.5 PreventBackHistoryMiddleware

Applied to the whole admin group. Sets a battery of security headers (X-Frame-Options DENY, nosniff, HSTS, no-cache so the back button can't show cached authed pages, CSP `frame-ancestors 'none'`) and a manual idle-timeout branch.

> **SECURITY — REVIEW THESE**
>
> 1. **Open self-registration:** `/register` is public and ungated; it creates a role-less active user. Such a user can't pass permission checks but is a valid authenticated account. Remove or gate for production.
> 2. **Password reset with no identity proof:** the public `/update-password` POST lets anyone set a new password for any existing email with no token/OTP/current-password check — a full account-takeover vector. This is*not* a real forgot-password flow (the employee portal's reset flow, by contrast, uses proper tokenized `Password::reset`).
> 3. **Permissive CORS with credentials:** the middleware reflects the caller's `Origin` back as `Access-Control-Allow-Origin` together with `Allow-Credentials: true` , which defeats same-origin protection for any browser that honors it.
> 4. **Minor:** GET logout is CSRF-triggerable; the plaintext password is flashed to session on failed login; the custom idle-timeout branch is effectively inert (nothing sets the `logged_in` session key it depends on, and the threshold is 3600s despite a "30 minutes" comment).

5. Module — Waste Disposal

Core operational domain: records of waste pumped & hauled, feeding the County of Hawai‘i monthly source report. Table: `waste_disposal_details` (model `WasteDisposalDetails` , `timestamps = false` , manual `inserted_at`).

5.1 Public intake (`Frontend/HomeController`)

- GET `/log-waste-disposal` — the public form; POST `/waste-entry` → `store_waste_entry()` .
- Validates dates as `m/d/Y` , generator/waste-type/address/volume/unit/zip, `discharge_site ∈ {Hilo, Kona}` , transporter initials, vehicle license. Creates the record with the submitter's IP and `inserted_at = now()` , then fires an `AdminNotification` to all admins (type `disposal`). No file uploads, no drafts.

5.2 Admin review & output (`Backend/DisposalController`)

Action	Behavior
`index()`	Filterable list (year / from-date / to-date), ordered by pickup date desc, with a distinct-year dropdown. *(The file you had open — `disposal/index.blade.php` — is this list: DataTable + row-select bulk PDF, CSV export, permission-gated delete.)*

`export()`	Streamed CSV (optionally by year): IP, pickup date, generator, waste type, address, volume, unit, zip, discharge date/site, transporter, vehicle license.
`exportSelectedPdf()`	Takes comma-joined selected IDs. One record → single dompdf download; multiple → builds a `ZipArchive` of per-record PDFs and streams it (`deleteFileAfterSend`).
`generate_monthly_report()`	Renders the official **County of Hawai'i Wastewater Sludge Pumping & Hauling Source Report** (A4 landscape PDF) for a chosen month/year with registration/permit/signatory fields.
`destroy()`	Soft-delete (restorable).

NOTE

`create/store/edit/update` routes are declared but **there is no `store` or `update` method** — disposal records are created only via the public form, not hand-entered by admins. The `index()` query is also built three times redundantly (dead reassignments).

6. Module — Cesspool & Septic Inspections

Two near-identical field-inspection modules. Tables `cesspool_system_details` / `septic_system_details` (both `timestamps = false`, soft-deletes). Each has a public intake form with **draft auto-save**, and an admin screen for review, media editing, PDF export and email delivery.

6.1 Public intake (`Frontend/CesspoolController` / `SepticController`)

- GET form pages first **purge expired drafts** (`is_draft=true AND expires_at < now`).
- POST `/…/draft` — AJAX draft save. Generates a UUID `session_key`, stores the row with `is_draft=true` and a **1-hour TTL** (`expires_at = now()+1h`), keeps the key in session.
- POST `/…-systems` — final submit. Deletes this session's draft, builds the record with `is_draft=false`, fires an `AdminNotification` (type `cesspool` / `septic`).
- **Enrichment on submit:** captures IP + parsed browser/OS/device (`parseBrowserInfo`) and geolocates the IP via the external `ip-api.com` free API (`getLocation` , 3s timeout). Checkbox groups are collapsed into comma-separated human-readable strings. The typed/drawn signature arrives as a `data:image/…;base64` URL and is decoded to a PNG under `public/<domain>/signatures/` .

TECH-DEBT

`parseBrowserInfo()` and `getLocation()` are copy-pasted across both controllers — candidates for a shared service. The synchronous `ip-api.com` call adds an external dependency and up to 3s latency to every public submit. Septic also carries a dead `use PDF;` import.

6.2 Admin review (`Backend/CesspoolRecordsController` / `SepticRecordsController`)

Action	Behavior

`index()`	Filter by date range + draft/submitted status. Cesspool orders by `id desc`; septic by `inserted_at desc`. Drafts are visible in the admin list.
`edit()` / `update()`	Edit record + media. **Image** → Laravel public storage disk (`cesspool|septic/images`). **Video** → written directly into `public/<domain>/videos/`. **Signature** → decoded base64 PNG into `public/<domain>/signatures/`. All support remove flags.
`exportPdf()`	A4-portrait inspection report via dompdf.
`sendReport()`	Renders the PDF in-memory and emails it with the branded `emails.inspection_report` template, attached. **Cesspool supports multiple recipients** (splits on whitespace/comma/semicolon, validates each); **septic accepts a single address only.**
`destroy()`	Soft-delete; uploaded files intentionally retained on disk.

7. Module — Document Management & E-Signature

Admins upload documents into folders (categories) and either publish them publicly or assign them to specific employees. Documents can require a mandatory "read & sign" acknowledgment that produces a certificate PDF.

7.1 Data model

- `DocumentCategory` — the "folder" (name, unique slug, is_active).
- `Document` — file metadata, `is_public`, `requires_acknowledgment`, `acknowledgment_due`. Assignment to employees is via the `document_user` pivot (`assignees()`). The old `documents.user_id` single-owner column is **legacy** — controllers now force it to `null` and route all assignment through the pivot.
- `DocumentAcknowledgment` — one signature record per employee per document (unique on `document_id,user_id`): signed name, IP, `signed_pdf_path` (the certificate), `acknowledged_at`.

7.2 Admin side (`DocumentController` / `DocumentCategoryController`)

- `store/update` — validate (mimes pdf/doc/docx/xls/xlsx/jpg/png, max from `config/uploads.php`), read file metadata *before* moving the temp file, store under `public/uploads/documents`. If not public, `sync()` assignees and notify only newly-added employees. **Read-and-sign can only be required on personal PDF documents** (enforced in `resolveRequiresAck` — throws a validation error otherwise).
- `signOffs()` — compliance dashboard: all sign-required docs with signed-vs-pending counts (a doc is "complete" when `acknowledgments ≥ assignees`).
- `acknowledgments(Document)` — per-document signer list (signed vs pending); `downloadSigned()` streams any employee's certificate.
- A folder with documents can't be deleted; destroying a document deletes its physical file then soft-deletes the row.

7.3 Employee read-and-sign flow (frontend)

1. Employee sees a document card; sign-required docs appear under "Action Required" (`pendingAcknowledgments`, ordered by due date).

2. GET `/document-library/sign/{document}` shows the read screen (must be logged in, assigned, and the doc must require ack).
3. POST acknowledge — validates typed `signed_name` + `agree` checkbox, captures IP + timestamp, calls `DocumentSignatureService::generateSignedPdf()`, creates the `DocumentAcknowledgment`, writes an ActivityLog entry, and notifies admins (type `document`).
4. **The original file stays locked** (403 on download) until the employee has signed.

7.4 DocumentSignatureService — how "signing" works

> **IMPORTANT CLARIFICATION**
> The service does **not** stamp/overlay the original PDF. It renders a **separate "Certificate of Acknowledgment" PDF** from the Blade view `pdf.acknowledgment_certificate` (embedding logo/watermark/seal as base64 data URIs). The **typed full name is the signature**; IP + timestamp are recorded on the certificate. Rendering **prefers headless Chrome/Edge** (`--headless=new --print-to-pdf`, discovered via `CHROME_PATH` or a probe list) for pixel-accurate CSS, and **falls back to dompdf** (A4 landscape) if no Chrome binary is found.

8. Module — Announcements

- `Announcement`: title, unique slug, body, optional image, `is_active`, `published_at`, `created_by`. Slug is generated on create (backfilled by migration for legacy rows) and never regenerated on update.
- Admin CRUD (`AnnouncementController`) with image upload under `public/uploads/announcements` and remove-image support; `published_at` defaults to now.
- **Public:** announcements are viewable by anyone without login — GET `/employee-announcements` (paginated) and `/employee-announcements/{slug}`. The employee dashboard shows the 3 latest via `scopePublishedLatest()`.

9. Module — Community Calendar

- `CalendarEvent`: title, description, start/end date, `all_day`, start/end time, `category`, location, optional `attachment` (flyer). A `CATEGORIES` constant maps 5 keys (holiday/meeting/training/pto/other) to labels + hex colors — this drives the legend, validation whitelist and event colors.
- Admin CRUD (`CalendarEventController`) with flyer upload (`public/uploads/calendar`, mimes pdf/doc/docx/images).
- **FullCalendar JSON feed** (`events()`): maps each active event to a FullCalendar object. All-day events add +1 day to `end` (FullCalendar treats all-day end as exclusive); timed events emit `date T time` strings. Category color becomes a soft translucent background (`rgba(...,0.14)`) with a solid border/text color via `hexToRgb()`.
- **Public read-only view** at `/community-calendar` + its own events feed (8 upcoming events + legend).

10. Module — Incident Reports

Workplace-incident reporting filed by employees or by managers on their behalf. Table `incident_reports` + `incident_report_photos` (which stores *both* images and videos, distinguished by MIME type).

- **Permission model:** `canManage() = hasPermission('incident-reports.edit')`. Managers see/manage all; non-managers (employees) are scoped to `reported_by = own id`.
- `reference_no` is auto-generated as `IR-<year>-<zero-padded id>`. Statuses: open / under-review / closed. Source: employee / admin.
- **Employee submit** (frontend) attaches up to 6 photos + 4 videos, sets `reported_by` = filer and `employee_id` = involved employee, and notifies all admins.
- **Admin** (`IncidentReportController`): list with filters + year dropdown, CSV export, create/edit, status changes stamp `reviewed_by` / `reviewed_at` . When a manager files/updates on behalf of an employee, the affected employee(s) get a `notifyUser` notification. Destroy is soft-delete; media files are deliberately kept (a single photo can also be soft-deleted individually).

11. Module — Employees (HR) & Employee Portal

11.1 Employee authentication

Employees are `users` with role slug `employee`, authenticated on the same `web` guard. On login the controller stores `employee_password_hash` in the session. The `EmployeeAuth` middleware checks (a) the user is logged in and is an employee, and (b) the stored hash still matches the DB password — if an admin or another device changed the password, the session is invalidated and the user is forced to re-login. The employee portal has a **proper tokenized password-reset flow** (`Password::sendResetLink` / `Password::reset`); `AppServiceProvider` overrides the reset URL to point at the employee frontend page.

11.2 Employee Portal features (`Frontend/EmployeesController` , 801 lines)

Area	Capability
Portal / dashboard	Public landing (no login) + authenticated dashboard showing latest announcements, the employee's own incident reports and status stats.
Profile & password	Edit name/phone/avatar (email intentionally not editable); change password (re-stores the session hash so the user isn't logged out mid-request).
Document library	Public + personal documents, read-and-sign flow, signed-certificate download (§7.3).
Incident reporting	Submit incidents with photos/videos (§10).
Announcements & calendar	Read-only views of published announcements and upcoming events.
Notifications	Bell feed + live toast polling (§12).

11.3 HR admin side (`Backend/EmployeeController`)

- Manages only `employee`-role users. `store()` creates the account and **sends a welcome email containing the plaintext password**, stamping `welcome_email_sent_at` on success. Mail failure does *not* roll back account creation (leaves the timestamp null for retry).

- `update()` has welcome-email retry logic: if never sent and a new password is provided, it resends; passwords are hashed and unreadable, so retry requires setting a fresh password.
- The edit screen also surfaces the employee's signed-document acknowledgments.

12. Notifications

- `AdminNotification` model with `recipient()` and `actor()` relations. Two creators: `notifyAdmins()` (bulk-inserts one row per active non-employee user) and `notifyUser()` (single recipient, e.g. an affected employee).
- **Created by:** public form submissions (disposal/cesspool/septic), document shares & signatures, and incident reports → admins; manager-filed/updated incidents → the affected employee.
- Both admin and employee UIs have a bell feed (view composers inject unread count + latest 8) plus a JSON `feed` endpoint polled for live toasts (client sends an `after` id so only new items replay), "mark read", and "read all". Note: `notifyAdmins` uses a bulk `insert`, bypassing model events/observers (harmless — no observer on this model).

13. Activity Log & Auditing

- `ActivityObserver` is attached (in `AppServiceProvider`) to 12 models: Document, DocumentCategory, Announcement, IncidentReport, CalendarEvent, User, Role, Permission, EmailSettingsDetails, WasteDisposalDetails, CesspoolSystemDetails, SepticSystemDetails. It records create/update/delete/restore with the changed-column list, the acting user, IP, URL and HTTP method. **Console/CLI operations are skipped** (migrations/seeders/tinker produce no logs).
- `ActivityLog::write()` is a safe static helper callable anywhere (used directly for login/logout and document-signing events).
- The admin UI splits the log into two views via `ActivityLogController`: **"Forms"** (Cesspool Records, Septic Records, Disposal Details) and **"HR"** (everything else), each with module/event/user/text/date filters.

14. Email & Scheduled Jobs

- **Transactional mail:** employee welcome email (with credentials), inspection-report delivery (PDF attached), password-reset links. All sent **synchronously** in-request (no queue), via SMTP.
- **Monthly report reminder:** command `email:monthly-report-reminder` emails the recipients configured in `EmailSettingsDetails` (default + up to 3 addresses) a reminder with an admin-login CTA.
- **Schedule:** defined in `routes/console.php` → `lastDayOfMonth('09:00')` in `Pacific/Honolulu`. Requires the system cron to run `php artisan schedule:run` every minute.

> **STALE CONFIG**
> `app/Console/Kernel.php` contains an old commented-out `everyMinute()` version with a misleading "runs every day at 9:00 AM" comment. It is inert — the authoritative schedule is in `console.php` (last day of month, 09:00 Hawai'i time). The command itself does not re-check the date; gating is entirely the scheduler's job.

15. Database Schema Summary

Table	Purpose
`users`	Staff + employees (role_id, is_active, welcome_email_sent_at, soft-delete + deleted_by)
`roles` / `permissions` / `permission_role`	RBAC (roles, permission catalog, pivot)
`waste_disposal_details`	Public waste-disposal submissions (⚠ no migration — see §17)
`cesspool_system_details` / `septic_system_details`	Inspection records (drafts, media, signatures, client/geo fingerprint)
`document_categories` / `documents` / `document_user` / `document_acknowledgments`	Folders, files, per-employee assignment pivot, signature certificates
`announcements`	Company announcements (slug added + backfilled by later migration)
`calendar_events`	Community calendar (category, all-day/timed, flyer attachment)
`incident_reports` / `incident_report_photos`	Incident reports + attachments (photos & videos in one table)
`activity_logs`	Audit trail
`admin_notifications`	In-app notifications (admin + employee)
`email_settings`	Monthly-report recipient configuration (`timestamps=false` , custom inserted/modified columns)
`jobs` / `cache`	Laravel infrastructure tables

16. File Storage Conventions

Most uploads go directly into `public/uploads/…` (documents, documents/signed certificates, avatars, incident-reports, announcements, calendar) and are served via `asset()`. Soft-deleting a row does *not* remove its file unless the controller explicitly unlinks it on `destroy` /replace.

> **STORAGE ASYMMETRY (EASY TO TRIP ON)**
> Cesspool/septic inspection **images** use the Laravel **public storage disk** (`storage/app/public/…` , which requires `php artisan storage:link`), while **videos and signatures** for the same records are written straight into `public/` . The PDF templates account for this by resolving images with `storage_path()` but signatures with `public_path()` . Video cannot be embedded in dompdf PDFs — it's rendered as a clickable link.

17. Known Issues, Risks & Handover Flags

Severity	Item

High	Public `/update-password` sets any user's password with no token/identity check (account takeover). Fix before any public exposure.
High	No migration exists for `waste_disposal_details`, yet the whole disposal module depends on it. A fresh `php artisan migrate` on an empty DB will NOT create it — the table is currently created out-of-band. Add a migration before any clean deploy/CI.
Medium	Open, ungated `/register` self-registration route on the admin app.
Medium	Reflected-origin CORS with `Allow-Credentials: true` in `PreventBackHistoryMiddleware`.
Medium	Synchronous `ip-api.com` geolocation call on every cesspool/septic public submit (external dependency + up to 3s latency). Consider making it async/optional.
Low	No automated test coverage (only the two default `ExampleTest` stubs).
Low	Duplicated `parseBrowserInfo` / `getLocation` across cesspool/septic controllers; `DisposalController::index` builds its query 3×; dead imports (`DisposalDetails`, `EmailSettingsDetails`, `PDF`); stale `Kernel.php` schedule comment.
Low	Repo hygiene: a ~6 MB `cron.log` and a stray `public/test.php` are committed; `.env` is tracked with `APP_DEBUG=true`.
Low	`EmailSettingsController::destroy` catches an unqualified `Exception` (not imported in that namespace) — a latent fatal if an exception is actually thrown.

18. Setup & Deployment Notes

1. `composer install` & `npm install`; copy `.env.example` → `.env`; `php artisan key:generate`. (A composer `setup` script bundles these.)
2. Create the MySQL database and run `php artisan migrate`. **Then provision `waste_disposal_details` manually** (no migration yet — see §17) and seed roles/permissions + a superadmin.
3. `php artisan storage:link` (required — cesspool/septic images live on the public storage disk).
4. Ensure `public/uploads/` and its subfolders are writable by the web server.
5. Configure SMTP in `.env`; populate `email_settings` for the monthly-report reminder.
6. Register the system cron: `* * * * * php artisan schedule:run` (drives the last-day-of-month reminder).
7. Optional: install Chrome/Edge (or set `CHROME_PATH`) for pixel-accurate signature certificates; otherwise dompdf is used automatically.
8. `npm run build` for production assets. In production set `APP_ENV=production`, `APP_DEBUG=false`, and resolve the §17 security items.

End of document — generated from a full static analysis of the `new-career-app` branch. Line-level references for every item summarized here are available in the source controllers, models, middleware and migrations cited throughout.